

Druid Prototype

Prior Reading

- Basics:
 - The [official Druid documentation](#) is the best resource before diving into this RFC.
 - The [stream-push](#) section of the Druid docs cover the mode of ingestion we plan to use (as opposed to [stream-pull](#)).
 - The [Strata Conference Slides](#) give a great overview, and have details about Yahoo's usage of it.
- Advanced reading:
 - The [Druid whitepaper](#); its co-author, [Eric Tschetter](#), visited us in November and is referred to as "Eric" in this RFC.
 - [Tranquility](#), a framework that integrates closely with Druid to support the guarantees we need, is referenced heavily.

Implementation Overview

High Level Goals

Our ads, analytics engineering (AE), and product analytics (PA) teams need a system that can quickly service queries for complex aggregations of Tumblr user actions. Druid is a specialized datastore that was designed for these types of queries.

This RFC covers the first of phase of its adoption: Setting up a prototype cluster for the AE and PA teams, and implementing the automation along the way. It explains why we chose Druid, how we intend to use it, and the questions we need to answer. After this phase, a second RFC will focus on its integration with our main HDFS cluster (CRUNCHY) and productionizing it for our most compelling use case: The Ads Preferred Marketing Developer (PMD) project.

Ads Preferred Marketing Developer (PMD) Project

The ads team is rolling out an API that our most important advertisers (ie. PMDs) will use to programmatically run and modify campaigns in real time. To make automated decisions, PMDs expect to make complicated queries about real-time ad impressions in under a second. For example, A PMD might wish to analyze impressions from users in Brazil that have reblogged their ad in the last hour using Firefox. They may post 3 ads, identify the one that has been reblogged the most in that region, and move their spend to

Sassan

Feb 8 2017

the most successful one. Platforms like Facebook and Twitter already provide this functionality (with 10 minute granularity). Druid will allow the ads team to support feature parity, and provide better granularity than our competitors do.

The most challenging aspect of these queries is our high-throughput impression data, which have [high cardinality](#). A single post impression might be associated with a user agent, geographic region, mobile device, length of membership, etc. We must not only track an ad itself, but any reblogs of it separately.

A PMD query, if it were in SQL, might look like this:

```
SELECT minute,
       ad_id,
       device_platform,
       device_type,
       sum(impressions) as impressions,
       sum(likes) as likes,
       sum(reblogs) as reblogs,
       sum(follows) as follows,
       sum(clicks) as clicks,
       average(clicks / impressions) as click_through_rate
FROM ads_analytics
WHERE ts BETWEEN 1486339200 AND 1486342800
      AND country in ('US', 'CA', 'GB')
      AND advertiser_id = 123456
GROUP BY minute,
         ad_id,
         device_platform,
         device_type
```

This use case is further complicated by late arriving events. For example, an impression event can originate from a mobile device that was offline when it occurred. When that device comes back online 4 hours later and submit the event, our pipeline has to correctly handle it.

Sassan
Feb 8 2017

Druid was designed to service these types of queries at Metamarket, and it was later adopted at Flurry following their acquisition by Yahoo, where Druid co-creator Eric Tschetter resides.

This use case is challenging to support operationally: Druid will be in the “user-facing” path (the user being the PMD), and downtime will have revenue impact. For this reason, we’re going to begin our adoption by first easing into a simpler use case: offline analytics queries by the AE and PA teams.

PA and Business Analytics

In addition to queries like in the above use case, the PA team has struggled with complex queries that don’t fit well with our Scribe-HDFS-MapReduce pipeline.

One common example from the PA team comes from the aggregation of metrics at varying levels of dimensionality and aggregation. Whenever new events are introduced and require daily tracking and reporting, the PA team must develop custom ETL queries and scripts to handle the event tracking at each level of aggregation and dimensionality. Because this requires custom effort, requests from leadership come which inevitably require different dimensionality (for example, not only device and geo, which we typically roll up on a daily basis, but also app version). At that point, the team is left writing custom queries to satisfy the request and weighing the relative costs of additional data storage requirements as well as the additional complexity in maintaining Crystal-ball against the benefits of reducing manual querying at a later date.

Our marketing team also hopes to leverage Druid in self-service fashion, in order to achieve behavioral analysis through the funnel, in a time-series based fashion (looking at period over period, correlations between different variables, etc), to answer tough questions like:

- How do we grow 2-day+ user retention?
- How do we grow incremental new registered users?
- How do we identify ways to notice and limit user churn?
- How do we identify ways to measure and improve user engagement?

Because the PA use case and their AE dependencies are not in the critical path, and can tolerate downtime, we will implement them first, so that we can apply those learnings to the ads use case later in the year.

Sassan

Feb 8 2017

Questions for the Next Phase

By setting up this prototype cluster, we hope to answer a number of important questions:

- What type of hardware will we need to support 95th percentile sub-second queries? If it is cost prohibitive, what type of query latencies are acceptable? Druid's performance is very dependent on the specific queries, so we might not be able to gage this for the PMD use case until we begin testing it in the next phase.
- What is the optimal configuration of each component? We'll use the PA use case to discover the bottlenecks.
- What is the optimal tiering configuration for our access patterns?
- How does Druid behave when its different components break - eg. broken indexer, overlord, HDFS is down, etc. For example, given that data is only stored on the real-time nodes/peons immediately after ingestion, and not flushed to historical until later, what happens when the realtime node/peon goes down? The [Druid FAQ](#) touches on some of these, and we'll dig deeper.
- Should we support ingestion mechanisms that don't involve Kafka-Tranquility stack, which don't make the same guarantees?
- What type of retention can our HDFS capacity support?
- How can minimize double-writing to HDFS while still supporting 1-day batch reconciliation?

Several questions will *not be* covered by the Druid RFCs, and need to be addressed separately:

- We don't yet have a standardized mechanism to push data into Kafka. LittleSister doesn't write unified schema into Kafka yet, and scribe topics are not available without Franz. Some producers, like the ads team, also [also plan to preprocess data](#) before Druid ingestion.
- What types of real-time ingestion/indexing latencies can the ads project tolerate?
- What are the update frequency expectations around batch processing?
- How will the application handle real-time ingestion failures? How much lag are clients willing to tolerate during this state?
- What type of retention does the PMD API need?

Known Issues and Limitations

Some of the important guarantees that we need from Druid (like exactly-once ingestion) require Kafka, either using Tranquility or the Kafka Indexing Service. This may rule out non-kafka ingestion mechanisms in production. This is particularly true for real-time nodes, which depend on replaying Kafka traffic in case of node failure.

Druid's handling of late arriving events is non-trivial: Once an event is past the buffer of the "windowPeriod", it will be dropped. This is especially problematic if, due to an incident, we're forced to replay buffered events that have missed the window period for late arriving events. Most deployments address this by using batch ingestion to backfill the data, which requires that write all of our Kafka

Sassan

Feb 8 2017

streams to HDFS, if only for this purpose. Although most of our logs require being written to HDFS anyway for Hive and MR access, others will redundantly increase HDFS usage. It'll also require automation to continuously backfill. Using the kafka indexing service may mitigate this.

There are three real-time ingestion mechanisms outlined in this RFC: real-time nodes, tranquility+indexing service, and the kafka indexing service. Of these three, real-time nodes are the least desirable for several reasons:

- Kafka topics that require multiple consumer groups [pose replication issues](#) that need more [complicated setups](#) to resolve.
- Data loss is possible if the disk on the realtime node dies (see the fault tolerance section in the [Druid Design docs](#)).
- Schema changes require restart of all realtime nodes (not a huge problem but might be annoying if we have to do it a lot)

Druid doesn't automatically allow us to identify and kill bad queries, so this is a manual process. For queries generated by our own API, we will need to vet them for correctness. If we allow our PMDs to execute arbitrary queries, they may be able to issue queries that degrade the cluster.

Implementation Details

Until our primary HDFS cluster (Crunchy) is upgraded to CDH5, we will set up a Druid cluster against the upgraded BF2_TESTING cluster. We'll do this in two parts:

1. Set up the components to support batch processing.
2. Add support for real-time ingestion with Kafka.

Druid has a complex topology made up of several node types - most of our development work will be mapping these nodetypes to container images, setting their roles via collins, and making them discoverable to clients in a way that's consistent with our infrastructure.

Druid Background

As covered in greater detail in prior readings, a key aspect of Druid's design is that offline data and streaming data is ingested and indexed using two separate pipelines:

1. Batch Ingestion: Data is loaded from HDFS using MR jobs.
2. Real-time Stream Ingestion:
 - a. [Stream-push](#) using Tranquility and the indexing service (Recommended)
 - b. [Stream-pull](#) using real-time nodes

Sassan
Feb 8 2017

Clients make queries to brokers, which then fan out requests to the real-time and deep storage nodes. Producers interact with Tranquility service instances, which are recommended to be co-located with the middle managers (see below).

We will need to deploy five new types of nodes to support the [Druid roles](#):

- Brokers (which take client traffic)
- Coordinator (with a hot/failover set)
- Historical Nodes
- Indexer service:
 - Overlord
 - Middle managers
- Real-time Nodes (Optional)

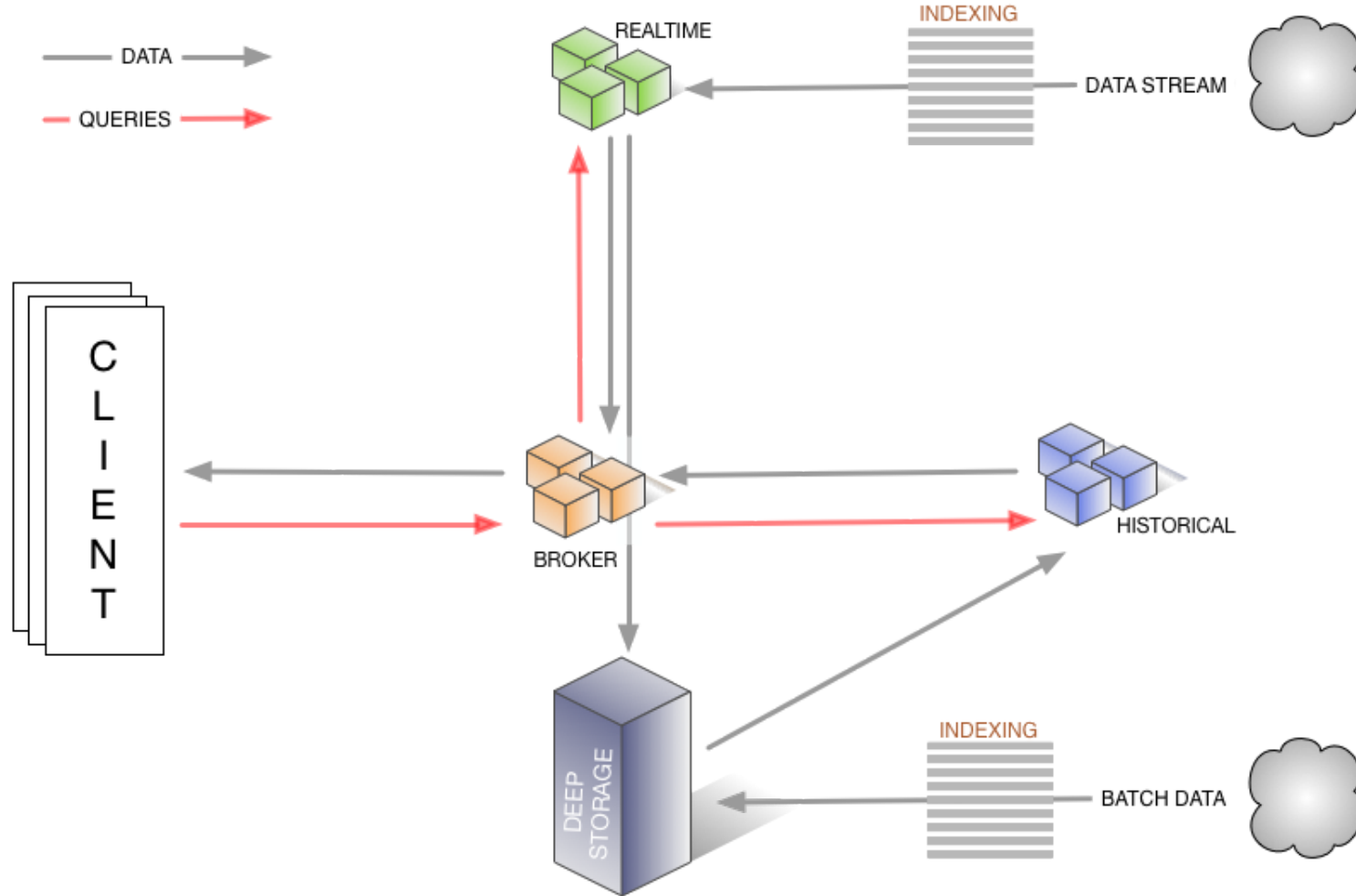


diagram-1

Diagram 1: A query from the end user will go to "Broker", which dispatches the query to real-time nodes and/or historical Nodes, and then combines the results.

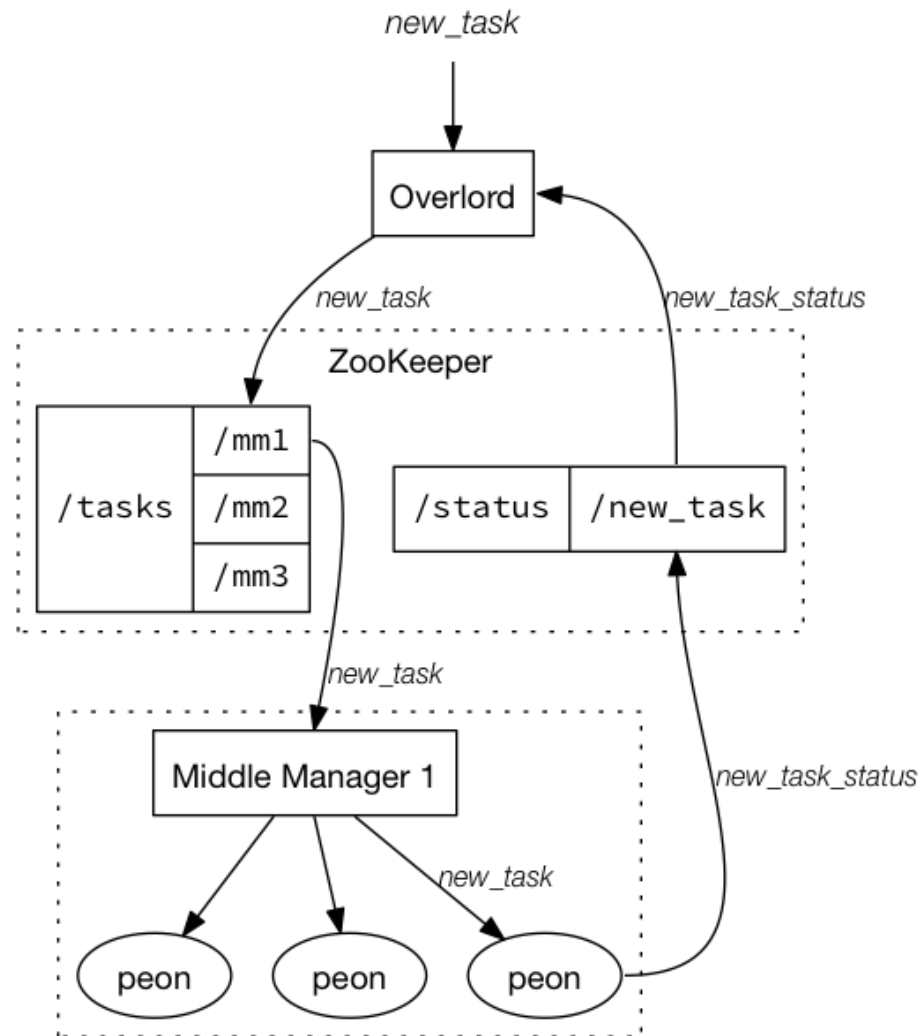


diagram-2

Diagram 2: A batch ingestion job is sent to one “Overlord”, which acts as a job manager.

Sassan

Feb 8 2017

Development Overview

We plan to configure Druid's external dependencies as follows:

- Use CDH5 HDFS cluster for deep storage. Initially this will be TESTING_BF2, and after the CDH5 upgrade, CRUNCHY itself. Druid is bundled with the Hadoop 2.3 client, but we will be using it with the Hadoop 2.6 libraries used by the CDH 5.6 .
- We'll use the PLATFORM_BF2 MySQL pool for Druid's metadata storage.
- Use a dedicated ZooKeeper cluster.

For batch indexing, Druid should allow us to ingest data directly from Crunchy, as it only requires CDH5 for its deep storage, not the upstream data. We'll begin testing by sampling the following datasets:

- The Kraken LittleSister events hive table
- [Flatted Ads Schemas](#)

After we've deployed batch indexing, we'll evaluate and choose one of two methods of real-time kafka ingestions:

1. [stream-push](#) and [tranquility](#)
2. [Kafka indexing service](#)

If we choose option #1, we'll colocate the Tranquility servers with the middle managers ([per recommendations](#)). Although Druid also supports the simpler [stream-pull](#) ingestion with real-time nodes, it doesn't scale as well, and has weaker guarantees (see the Known Limitations section).

Option #2 is a cleaner method of supporting real-time ingestion with better support for late arriving events, but is more recent. However Eric and Himanshu have given it their blessings, and it looks like the likely candidate.

Using either approach, we'll start by ingesting data from GRETE, our production kafka cluster. In later phases we may set up a dedicated Kafka cluster for the ads use case.

Both options will also likely require at least 1 day of writing our Kafka data to HDFS for batch reconciliation.

Segment Configuration & Retention Policies

Druid provides a configurable [retention rules](#) that we'll set aggressively so that TESTING_BF2's low capacity can accommodate our testing. For example, we'll drop everything older than 1 month with the following rules:

Sassan

Feb 8 2017

- LoadByPeriod P1M
- DropByPeriod P1M

This mechanism removes data from Druid and marks segment as unused in the metadata, but doesn't remove the underlying raw data from HDFS. [Per the Druid forums](#) we'll need to schedule [kill tasks](#). We may make this a regularly scheduled job. A slight complication of this is that the retention rules need to be updated when selectively re-loading data from HDFS.

Once we're testing against CRUNCHY, we'll set these rules per input source, and tweak the replication factor across the historical node tiers.

A couple of special configuration tweaks we'll use for the segments in general:

- It's recommended that windowPeriod < segmentGranularity to prevent buildup of segments on the nodes.
- If we interact with Tranquility directly, the segmentGranularity and windowPeriod will be set in its configuration, and it passes them onto Druid.

Container Deployment

Given Tumblr's renewed focus on containerization and Druid's *relatively* stateless services, we will be using containers to sidestep building complex puppet automation and RPM-building. Until Tumblr reaches consensus on a [cluster manager](#), we'll use the light [docker::app puppet module](#) that deploys the container onto a bare metal host, and rely on host networking.

To make our prototyping easier until we have CI for containers, instead of setting up a separate image for each node type, we'll combine all the services into a single image. Within the container, we'll wrap the standard scripts to pick up the configurations from environment variables, so that we can pass them in via the container invocation. We'll initially start with the Druid committer base images, but we may want to eventually use a Redhat-based base image for consistency with SYS and other teams.

We'll use puppet to deploy indigo (and any other supporting services) onto the host, and drop light scripts to allow indigo to query the Druid HTTP metrics endpoint.

Because Druid is not *completely* stateless, we won't be use Marathon to manage the cluster. Our approach will let us easily migrate when Tumblr settles on a cluster manager and how our service discovery [should integrate with it](#). If that happens *after* the PMD API is out of beta (end of Q2), it will be a more significant migration.

Sassan

Feb 8 2017

Collins Integration and Discovery

Brokers will be fronted with round-robin DNS for clients to discover them with. The overlord will be fronted by a CNAME, for discovery by producers (like Tranquility) and batch job submissions. The coordinator will not need a CNAME, as it locates the historical nodes via ZooKeeper. If we choose to have clients interact directly with the Tranquility server, and not through Kafka alone, we'll investigate fronting that with round-robin DNS too, though the middle manager nodes may change frequently.

Nodes will be tagged in Collins with a *primary_role* of "DRUID" and logical secondary roles like "INDEXER".

Config Pushes & Operational Tasks

For critical nodes like the overlord, we'll use hot standbys that will be swapped in with CNAME changes. The DNS propagation time is bound by the configrr update time, which takes 10-15m, though they can be initiated manually. Once updated, our internal DNS converges almost immediately.

For internal druid nodes, we're still working through the cleanest way for them to discover one another, and how this will mesh with our containerized approach. Some questions we'll answer as we build the automation:

- Does Druid [properly respect DNS TTLs](#)? If not, will CNAME changes require restarts of the Druid processes?
- How do we swap out tranquility service instances if they're co-located with middle managers?
- How will we trigger configuration changes? Ideally all configuration will be passed into the container invocation, but the complex configs might make this hard. If we rebuild the images for each config change, how will we automate that? Or Should we mount an external config, or use a system like etcd/consul to trigger updates externally?
- Will our puppet automation triggers restarts?
- Will we maintain standbys for the different roles, or use automation to deploy the right service onto "generic" spares?

Tranquility config changes might be particularly tricky: We believe that changes to the kafka consumers (both RT nodes and tranquility) have to be made to the whole cluster at once. Having nodes with mixed configuration (which may happen during configuration updates) can cause the same data to be written to two segments. If true, this will require some tricky restart automation.

TODO: Mention how kafka topology changes are handled.

TODO: Mention how the configuration changes for new topics are deployed gradually to the cluster.

Sassan

Feb 8 2017

Alternate Solutions

Our MySQL infrastructure is our most reliable datastore, and comes with a ton of supporting like daily backups, sophisticated automation, and a path to multi-DC. Druid is a complex system that hasn't yet reached the maturity of MySQL, and we have every technical incentive to use MySQL if it were possible to service the PMD project using it.

Unfortunately, the relational model is a particularly bad fit for maintaining windowed metrics with arbitrary dimensions. For example, [in the most minimal ads implementation](#), a single impression event associated with a location, platform and post_id, and be a part of a 1m, 60m and 24h windows will amplify into 9 row updates. For the dashboard, approximately 10% of posts would be ad impressions that would incur these updates. With sophisticated caching and and custom logic we could handle our initial use cases, but the throughput becomes overwhelming with just a few more dimensions, and would not scale well.

The MySQL community is actively working on new engines designed for high-write/low-read patterns like this. The most promising of these is RocksDB, which FaceBook uses to power the timeline. Unfortunately MyRocks, the MySQL integration for RocksDB, isn't production-ready yet, and using RocksDB directly with clients is complicated and loses all the benefits of using MySQL in the first place.

Client Overview

Android Considerations

N/A

iOS Considerations

N/A

Mobile Web Considerations

N/A

Web Considerations

N/A

Sassan

Feb 8 2017

Infrastructure Considerations

Though not a factor for the prototype cluster, the production cluster from the next phase may warrant new hardware profiles we don't currently have:

- Brokers: Because these nodes are client-facing, the CPU and RAM these nodes are equipped with can have a big impact on query latencies.
- Historical nodes: Because these cache segments from HDFS to local disk and mmap them, they're recommended to have SSDs, and generous memory.

We're planning to reach out to Eric to find out which [specific Yahoo hardware profiles](#) Flurry uses, and get more details on their specific clusters.

Data Considerations

By experimenting with the prototype cluster, we will assess the impact of Druid on the systems it depends on:

- The ZooKeeper cluster used for internal cluster service discovery and maintaining data topology (via the coordinator).
- The MySQL cluster used for metadata, which stores data such as:
 - Segments metadata (both current segments and deleted segments)
 - Segment storage allocation rules
 - Config table (runtime configuration objects, not heavily used)
 - Task-related tables (used by the indexing service)
- Its effect on HDFS, which we expect to be significant.
- Its impact on the Kafka cluster.
- Queries to collins (unlikely).

We'll initially configure Druid against non-production clusters of all of these systems, so we expect no impact yet. Once we start experimenting against Crunchy, we'll have more insight.

API and Services Considerations

N/A

Sassan

Feb 8 2017

Monitoring, Logging, & Analytics Considerations

Our goal in this phase of the project is to identify the most helpful metrics we'll use to productionize Druid. We plan to feed into TSDB by using an indigo collector on the host that reads metrics via [Druid's HTTP metrics endpoint](#). We'll reach out to Eric to find out recommended metrics to watch and alert on.

Security Considerations

Until we flesh out security and authentication for Hadoop services via Kerberos, we don't plan to lock down Druid. We'll address security and authentication if and when we tackle it as part of our Kerberos efforts.

Testing Considerations

N/A

Performance Considerations

We will be ingesting data from Kafka, which is out of line with the app, so there is no performance impact.

Accessibility Considerations

N/A

Community Safety Considerations

N/A

Cost Considerations

By setting up the prototype cluster, we hope to get more insight into how much hardware we'll need to guarantee specific ingestion and query latencies. The cost of this hardware will guide the thresholds that we can support.

Until we move to a proper cluster manager, our cluster will not be elastic. Elasticity has limited benefits at Tumblr: Since we acquire our own hardware up-front, if we have the nodes available we might as well utilize them. If we don't, acquiring the nodes is a long

Sassan

Feb 8 2017

process that requires up-front analysis. One possible way we could leverage elasticity would be to spawn short-term clusters for dedicated use cases - we'll consider these use cases and its cost considerations in later phases of our work.

Multi-DC Considerations

When visiting our office, Eric mentioned that Flurry has deployed a failover mechanism where Druid clusters mirror one another in other DCs, and can be failed over to. Presumably Flurry relies on replicating their Kafka streams to other DCs to do this. We're planning to reach out to Eric to find out more details.

Our naive CNAME and round-robin approach of service discovery may pose issues in a multi-DC scenarios, where DNS will not converge as quickly as inside of the DC.

Other Topics:

N/A